

Problem 1: Understanding Numpy Broadcasting

a) Sum (+) with a Column Vector (2 points)

- Create a 2D array A with a shape of (4, 3).
- Create a 2D array B with a shape of (3, 1) using `np.array([[0],[1],[2]])`.
- Sum A and B using the + operator.
- What is the shape of the resulting array? Hint: This will produce an error. Explain why the broadcasting fails in this case.

```
import numpy as np
```

[33]

Python

a)

```
A=np.random.randn(4,3)
print(A)
B=np.array([[0],[1],[2]])
print(B)
```

[34]

Python

```
... [[-0.13750395 -1.04597421 -0.5915424 ]
      [-0.83486974  1.10159128  1.36005033]
      [-0.15803821 -1.57272468 -0.56213076]
      [ 2.15108121 -1.73612434  0.62063371]]
[[0]
 [1]
 [2]]
```

```
Qa=A+B
```

Python

```
-----  
ValueError                                Traceback (most recent call last)  
Cell In[35], line 1  
----> 1 Qa=A+B  
  
ValueError: operands could not be broadcast together with shapes (4,3) (3,1)
```

This part produce an error as the following:

```
ValueError Traceback (most recent call last)  
Cell In[6], line 1  
----> 1 Qa=A+B  
  
ValueError: operands could not be broadcast together with shapes (4,3) (3,1)
```

#read this documentation: <https://numpy.org/doc/stable/user/basics.broadcasting.html> (Two dimensions are compatible when they are equal, or one of them is 1)

This sum will produce a error and broadcasting fails in this condition as the shape of A is (4,3) and B is (3,1).

(4,3)
(3,1)

If we start with the right most dimensions 3 and 1 --> compatible as one of them is 1

The left dimension is 4 and 3 --> they are not equal and one of them is not 1

So one of the pairs fail--> broadcasting not possible.

b)

b) Sum (+) with a Row Vector (2 points)

- Create a 2D array A with a shape of (4, 3).
- Create a 2D array C with a shape of (1, 3) using `np.array([[0, 1, 2]])`.
- Sum A and C using the + operator.
- What is the shape of the resulting array? How to interpret the result?

```
A=np.random.randn(4,3)
print(A)
C=np.array([[0, 1, 2]])
print(C)
```

Python

```
[[ 0.95853416 -1.33655695  1.09174694]
 [-1.03876889  0.7196968  -0.7127034 ]
 [-0.08548708 -0.41174499 -2.23313801]
 [ 0.5320542   0.13747036  0.33977179]]
[[0 1 2]]
```

```
Qb=A+C
print(Qb)
print(np.shape(Qb))
```

Python

```
[[ 0.95853416 -0.33655695  3.09174694]
 [-1.03876889  1.7196968   1.2872966 ]
 [-0.08548708  0.58825501 -0.23313801]
 [ 0.5320542   1.13747036  2.33977179]]
(4, 3)
```

Shape is --> (4,3)

The broadcasting is successful as we have a (4,3) array being added to a (1,3) array.

(4,3)

(1,3)

comparing from right, 3 and 3 are equal --> compatible

left dimensions--> one of them is 1 --> compatible (dimension 1 can be expanded to 4)

(4,3)+(1,3)=(4,3)

```
[[ 0.95853416 -1.33655695  1.09174694]
 [-1.03876889  0.7196968  -0.7127034 ]
 [-0.08548708 -0.41174499 -2.23313801]
 [ 0.5320542   0.13747036  0.33977179]]
```

+

```
[0 1 2]
```

```
[0 1 2]
```

```
[0 1 2]
```

C)

c) Sum (+) with a 1D Array (2 points)

- Create a 2D array A with a shape of (3, 3).
- Create a 1D array D using `np.array([0, 1, 2])`.
- Sum A by D using the + operator.
- What is the shape of the resulting array? How does broadcast allows the sum between these arrays, given that they originally have different shapes?

```
A=np.random.randn(3,3)
print(A)
D=np.array([0, 1, 2])
print(D)
print(np.shape(D))
```

[36]

Python

```
... [[ 0.70169902  2.14678016 -0.46985677]
      [ 1.46647429  0.46756827  0.45634074]
      [-0.36814959  0.67047591 -0.02084464]]
[0 1 2]
(3,)
```

```
Qc=A+D
print(Qc)
print(np.shape(Qc))
```

[37]

Python

```
... [[ 0.70169902  3.14678016  1.53014323]
      [ 1.46647429  1.46756827  2.45634074]
      [-0.36814959  1.67047591  1.97915536]]
(3, 3)
```

shape is --> (3,3)

The broadcasting is successful as we have a (3,3) array being added to a 1d array (3,).

(3,3)

(3,) (1-dim and 3 elements)

comparing from right, 3 and 3(only 3 elements in array D) are equal --> compatible

left dimensions--> (3,) has no other dimension --> broadcast is done across the rows.

(3,3)+(3,)=(3,3)

```
[[ 0.70169902  3.14678016  1.53014323]
```

```
[ 1.46647429  1.46756827  2.45634074]
```

```
[-0.36814959  1.67047591  1.97915536]]
```

+

```
[0 1 2]
```

```
[0 1 2]
```

```
[0 1 2]
```


Problem 2: MLP Forward Propagation with NumPy

- a detailed docstring that describes its parameters
- the expected shapes for each argument

a) Implementation of Forward Propagation Functions

1. Relu(z): This function takes an input vector and, for each of its elements, returns the maximum between the element and zero.

```
[1] import numpy as np
✓ 0.1s Python
```

```
> ...
...
this function computes the relu activation function
parameters: z (input nd array)
returns: output nd array after applying relu.
...
def ReLU(z):
|     return np.maximum(0,z)
```

```
[3] ✓ 0.0s Python
```

2. forwardPropA(W1, b1, w2, b2, x): This function should compute the forward pass for a single input vector x using left multiplication convention for weights, i.e. the same used in our class.

```
...
this function computes the forward prop for a single input vector x using left multiplication for weights
parameters:
  W1 is weight of first layer, shape=(n_hidden, n_input)
  w2 is weight of second layer, shape=(n_output, n_hidden)
  b1 is the bias of first layer, shape=(n_hidden,1)
  b2 is the bias of second layer, shape=(n_output,1)
  x is the single input vector, shape=(n_input,1)
returns: output array after doing forward prop
...
```

```
def forwardPropA(W1, b1, w2, b2, x): |
...
  z1=W1@x+b1
  # (n_hidden,1)=(n_hidden, n_input) @ (n_input,1) + (n_hidden,1)
  h1=ReLU(z1)
  # (n_hidden,1)
  z2=w2@h1+b2
  # (n_output,1)=(n_output, n_hidden) @ (n_hidden,1) + (n_output,1)
  h2=ReLU(z2)
  # (n_output,1)
  return h2
```

```
✓ 0.0s Python
```

3)forwardPropB(W1, b1, w2, b2, X): Extend forwardPropA to handle a dataset X. This function should also use left multiplication.

```
'''
this function computes the forward prop for a dataset X using left multiplication for weights
parameters:
    W1 is weight of first layer, shape=(n_hidden, n_input)
    w2 is weight of second layer, shape=(n_output, n_hidden)
    b1 is the bias of first layer, shape=(n_hidden,1)
    b2 is the bias of second layer, shape=(n_output,1)
    X is the dataset, shape=(n_input,m)
returns: output array after doing forward prop
'''

def forwardPropB(W1, b1, w2, b2, X):

    z1=W1@X+b1
    #(n_hidden,m)=(n_hidden, n_input) @ (n_input,m) + (n_hidden,1)
    h1=ReLU(z1)
    #(n_hidden,m)

    z2=w2@h1+b2
    #(n_output,m)=(n_output, n_hidden) @ (n_hidden,m) + (n_output,1)
    h2=ReLU(z2)
    #(n_output,m)

    return h2
```

Python

4.forwardPropC(W1, b1, w2, b2, x): This function should compute the forward pass for a single input vector x using right multiplication convention for weight.

```
'''
this function computes the forward prop for a single input vector x using right multiplication for weights
parameters:
    W1 is weight of first layer, shape=(n_input, n_hidden)
    w2 is weight of second layer, shape=(n_hidden,n_output)
    b1 is the bias of first layer, shape=(1,n_hidden)
    b2 is the bias of second layer, shape=(1,n_output)
    x is the single input vector, shape=(1,n_input) * x is transposed from the previous x definition that had shape of (n_input,1)
returns: output array after doing forward prop
'''

def forwardPropC(W1, b1, w2, b2, x):

    z1=x@W1+b1
    #(1,n_hidden)=(1,n_input) @ (n_input, n_hidden) + (1,n_hidden)
    h1=ReLU(z1)
    #(1,n_hidden)

    z2=h1@w2+b2
    #(1,n_output)=(1,n_hidden) @ (n_hidden, n_output) + (1,n_output)
    h2=ReLU(z2)
    return h2
    #(1,n_output)
```

Python

5. forwardPropD(W1, b1, w2, b2, X): Extend forwardPropC to handle a dataset X. This function should also use right multiplication.

```
'''
this function computes the forward prop for a dataset X using right multiplication for weights
parameters:
    W1 is weight of first layer, shape=(n_input, n_hidden)
    w2 is weight of second layer, shape=(n_hidden, n_output)
    b1 is the bias of first layer, shape=(1, n_hidden)
    b2 is the bias of second layer, shape=(1, n_output)
    X is the dataset, shape=(m, n_input) * X is transposed from the previous X definition that had shape of (n_input, m)
returns: output array after doing forward prop
'''

def forwardPropD(W1, b1, w2, b2, X):

    z1=X@W1+b1
    #(m, n_hidden) = (m, n_input) @ (n_input, n_hidden) + (1, n_hidden)
    h1=ReLU(z1)
    #(m, n_hidden)

    z2=h1@w2+b2
    #(m, n_output) = (m, n_hidden) @ (n_hidden, n_output) + (1, n_output)
    h2=ReLU(z2)
    return h2
    #(m, n_output)
```

b) Applying the Functions to a Dataset

Selected forwardprop A and B

```
def forwardPropA(W1, b1, w2, b2, x):
    z1=W1@x+b1
    #(n_hidden,1)=(n_hidden, n_input) @ (n_input,1) + (n_hidden,1)
    h1=ReLU(z1)
    #(n_hidden,1)
    z2=w2@h1+b2
    #(n_output,1)=(n_output, n_hidden) @ (n_hidden,1) + (n_output,1)
    h2=ReLU(z2)
    #(n_output,1)

    return h2

def forwardPropB(W1, b1, w2, b2, X):
    z1=W1@X+b1
    #(n_hidden,m)=(n_hidden, n_input) @ (n_input,m) + (n_hidden,1)
    h1=ReLU(z1)
    #(n_hidden,m)
    z2=w2@h1+b2
    #(n_output,m)=(n_output, n_hidden) @ (n_hidden,m) + (n_output,1)
    h2=ReLU(z2)
    #(n_output,m)

    return h2
```

```
def forwardPropA(W1, b1, w2, b2, x):  
    z1=W1@x+b1  
    # (n_hidden,1)=(n_hidden, n_input) @ (n_input,1) + (n_hidden,1)  
    # (10,1)=(10, 5) @ (5,1) + (10,1)  
    h1=ReLU(z1)  
    # (n_hidden,1)  
    # (10,1)  
  
    z2=w2@h1+b2  
    # (n_output,1)=(n_output, n_hidden) @ (n_hidden,1) + (n_output,1)  
    # (1,1)=(1, 10) @ (10,1) + (1,1)  
    h2=ReLU(z2)  
    # (1,1)  
  
    return h2  
  
def forwardPropB(W1, b1, w2, b2, X):  
    z1=W1@X+b1  
    # (n_hidden,m)=(n_hidden, n_input) @ (n_input,m) + (n_hidden,1)  
    # (10,100)=(10, 5) @ (5,100) + (10,1)  
    h1=ReLU(z1)  
    # (n_hidden,m)  
    # (10,100)  
  
    z2=w2@h1+b2  
    # (n_output,m)=(n_output, n_hidden) @ (n_hidden,m) + (n_output,1)  
    # (1,100)=(1, 10) @ (10,100) + (1,1)  
    h2=ReLU(z2)  
    # (n_output,m)  
    # (1,100)  
  
    return h2
```

When we are working with a single input vector x , but want to go over the whole dataset X (100 observations in this example), we need to do a loop so it processes 100 different x vectors. So we would have our function definition of `forwardProp(W1,b1,w2,b2,x)` but we have a for loop that says for sample i , go through each of them one by one (we take one sample from dataset X each time) and pass it through the neural network to get the output instead of using vectorized X (all samples at once).

c) Performance Argument

The second function takes the whole dataset X and it should be faster as it takes all data (vectorization process of matrices) at once other than running a loop repeatedly to get to the whole dataset. In the second function X (whole dataset), we have the vectorized version that we provide the whole X to the NN instead of using a for loop.

Problem 3 - Linear Regression via Gradient Descent

$$a) f_{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

$$\text{where } \hat{y} = b + w_1 x_1 + \dots + w_m x_m = b + \sum_{i=1}^m w_i x_i$$

$$\Rightarrow \frac{\delta f_{MSE}}{\delta w_j} = \frac{2x_j}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

$$b) \frac{\delta f_{MSE}}{\delta w} = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_i$$

$$= \frac{2}{n} x^T (\hat{y} - y)$$

$$= \frac{2}{n} (X^T - I)^T y$$

$$c) \text{ Learning Rate} = 2 \times 10^{-4}$$

$$\text{Training Loss } D_{tr} = 297.1$$

$$\text{Test Loss } D_{te} = 312.3$$

d) Change the gradient descent function to also track loss on the test set. Stop gradient descent if the test loss stops decreasing.

Problem 3 Solution

In [16]:

```
import numpy as np
import matplotlib.pyplot as plt
import os
```

```
def gradient_descent(x_train, y_train, w0, alpha, num_steps):
    """
    Performs gradient descent to learn the weights of a linear regression model.

    Args:
        x_train (np.ndarray): The training input features.
        y_train (np.ndarray): The training target values.
        w0 (np.ndarray): The initial weights.
        alpha (float): The learning rate.
        num_steps (int): The number of gradient descent steps.

    Returns:
        np.ndarray: The learned weights.
    """
    # PRO TIP: CREATE DEEP COPY of arrays passed as args to avoid changing them
    # Your code here (~2 lines)
    x_train_copy = np.copy(x_train)
    y_train_copy = np.copy(y_train)
    w0 = np.copy(w0)

    losses = []

    for i in range(num_steps):

        # compute loss (~2 lines)
        # Your code here
        y_hat = np.dot(x_train_copy, w0)
        loss = np.mean((y_hat - y_train_copy) ** 2)
        losses.append(loss)

        # Placeholder for loss, remove after implementing
        # loss = 0
        print(f'Training loss = {loss:.3f}')

        # compute gradients and update params (~2 lines)
        # Your code here
        gradient = (2 / x_train_copy.shape[0]) * np.dot(x_train_copy.T, (y_hat - y_train_copy))
        w0 -= alpha * gradient
        w = w0

    return w, losses
```

```
def train_linear_regression(x_train, y_train, alpha=0.01):
    """
    Initializes and trains a linear regression model.

    Args:
        x_train (np.ndarray): The training input features.
        y_train (np.ndarray): The training target values.

    Returns:
        np.ndarray: The trained model weights.
    """
    # PRO TIP: setting a random seed makes the experiments reproducible (hopefully in
    np.random.seed(541)

    # Initialize weights (~2 lines)
    # Your code here

    w0 = np.random.randn(x_train.shape[1])
    # b = np.random.randn(1)

    # Placeholder for w0, remove after implementing
    # w0 = np.zeros(x_train.shape[1])
    print(f'Model coefficients: {np.round(w0,3)}')

    # Make initial prediction (~1 line)
    # Your code here
    y_pred = np.dot(x_train, w0)

    # Placeholder for y_pred, remove after implementing
    # y_pred = np.zeros(y_train.shape)
    print(y_pred[:5])

    # Calculate initial loss (~1-2 lines)
    # PRO TIP: tracking the train loss at the beginning and during optimization is
    # key for debugging, determining convergence
    # Your code here
    loss = np.mean((y_pred - y_train) ** 2)

    # Placeholder for loss, remove after implementing
    # loss = 0
    print(f'Training loss = {loss:.3f}')

    # Run gradient descent
    w, losses = gradient_descent(x_train, y_train, w0, alpha, 100)

    return w, losses
```



```
def plot_loss_curves(x_train: np.ndarray, y_train: np.ndarray, alphas: list[float]):  
    """  
    Plots the loss curves for different learning rates.  
  
    Args:  
        alphas (list of float): The learning rates to evaluate.  
    """  
    plt.figure(figsize=(10, 6))  
    for alpha in alphas:  
        # Train model with the given alpha  
        w, losses = train_linear_regression(x_train, y_train, alpha=alpha)  
        plt.plot(losses, label=f'alpha={alpha}')  
  
    plt.title('Loss Curves for Different Learning Rates')  
    plt.xlabel('Iteration')  
    plt.ylabel('Loss')  
    plt.legend()  
    plt.grid()  
    plt.show()  
    plt.savefig('loss_curves.pdf')  
  
def load_data():  
    """  
    Loads the training and testing data from .npy files.  
  
    Returns:  
        tuple: A tuple containing the training and testing data.  
    """  
    try:  
        X_tr = np.reshape(np.load("age_regression/age_regression_Xtr.npy"), (-1, 48*48))  
        ytr = np.load("age_regression/age_regression_ytr.npy")  
        X_te = np.reshape(np.load("age_regression/age_regression_Xte.npy"), (-1, 48*48))  
        yte = np.load("age_regression/age_regression_yte.npy")  
        print(X_tr.shape, ytr.shape, X_te.shape, yte.shape)  
    except FileNotFoundError:  
        print("Dataset files not found. Please download them and place them in the same c  
        return  
  
    return X_tr, ytr, X_te, yte
```



```
def main():
    """
    Main function to load data, pre-process, train, and evaluate the model.
    """
    # Load data from files
    # Make sure you have the .npy files in the same directory as this script
    X_tr, ytr, X_te, yte = load_data()

    # (optional) Pre-processing

    # PRO TIP: center and normalize the features from training set (~4 lines)
    # Your code here

    print("Normalizing the features...")
    # train_mean = np.mean(X_tr, axis=0)
    # train_std = np.std(X_tr, axis=0)
    # train_std[train_std == 0] = 1.0

    # X_tr = (X_tr - train_mean) / train_std
    # X_te = (X_te - train_mean) / train_std
    print("Feature normalization complete.")

    # PRO TIP: prepend a 1 to each of the observations in X_tr and X_te (~2 lines)
    # This way you can treat the bias just as a regular weight
    # Your code here
    print("Prepending a 1 to each observation in X_tr and X_te...")
    X_tr = np.hstack((np.ones((X_tr.shape[0], 1)), X_tr))
    X_te = np.hstack((np.ones((X_te.shape[0], 1)), X_te))
    print("Prepending complete.")
    print(f'X_tr shape after prepending: {X_tr.shape}, X_te shape after prepending: {X_te.shape}')

    # Train model
    print("Training the linear regression model...")
    w, losses = train_linear_regression(X_tr, ytr, alpha=2e-4)

    # Report fMSE cost on the training and testing data (separately)
    # Your code here (~4-6 lines)
    train_loss = np.mean((np.dot(X_tr, w) - ytr) ** 2)
    test_loss = np.mean((np.dot(X_te, w) - yte) ** 2)
    print(f'Training loss after training = {train_loss:.3f}')
    print(f'Testing loss after training = {test_loss:.3f}')
    # plot_loss_curves(X_tr, ytr, alphas=[1e-4, 2e-4, 5e-5])

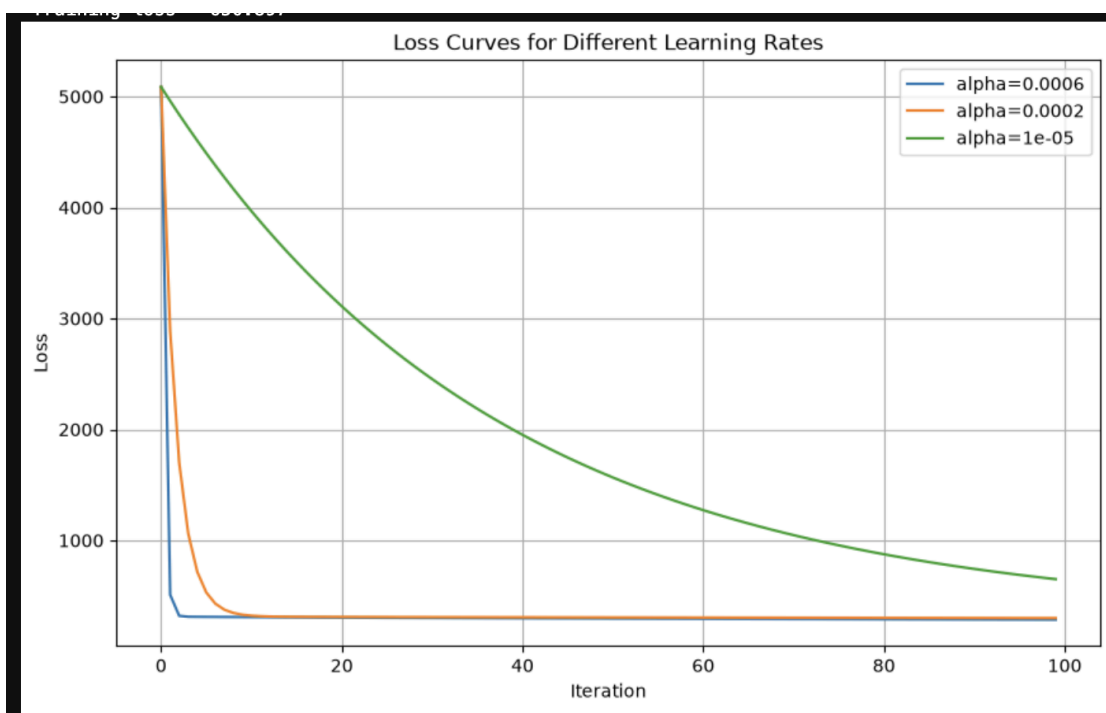
main()
```

```
(5000, 2304) (5000,) (2500, 2304) (2500,)
Normalizing the features...
Feature normalization complete.
Prepending a 1 to each observation in X_tr and X_te...
Prepending complete.
X_tr shape after prepending: (5000, 2305), X_te shape after prepending: (2500, 2305)
Training the linear regression model...
Model coefficients: [ 0.803  0.039  0.109 ...  0.882 -1.13 -0.027]
[-36.12891776 -26.73343558 -29.29599607 -23.1388821 -25.61559408]
Training loss = 5066.104
Training loss = 5066.104
Training loss = 2873.065
Training loss = 1691.228
Training loss = 1054.294
Training loss = 710.991
Training loss = 525.917
Training loss = 426.108
Training loss = 372.246
Training loss = 343.144
Training loss = 327.384
Training loss = 318.815
Training loss = 314.120
Training loss = 311.515
Training loss = 310.035
Training loss = 309.163
Training loss = 308.618
Training loss = 308.250
Training loss = 307.978
Training loss = 307.758
Training loss = 307.567
Training loss = 307.391
Training loss = 307.224
Training loss = 307.063
Training loss = 306.904
Training loss = 306.748
Training loss = 306.594
Training loss = 306.440
Training loss = 306.288
```

```
Training loss = 297.978
Training loss = 297.870
Training loss = 297.763
Training loss = 297.657
Training loss = 297.551
Training loss = 297.445
Training loss = 297.340
Training loss = 297.235
Training loss after training = 297.131
Testing loss after training = 312.245
```

```
: X_tr, ytr, X_te, yte = load_data()
  plot_loss_curves(X_tr, ytr, alphas=[6e-4, 2e-4, 1e-5])

(5000, 2304) (5000,) (2500, 2304) (2500,)
Model coefficients: [ 0.803  0.039  0.109 ... -0.098  0.882 -1.13 ]
[-33.81815081 -25.70298082 -32.41025507 -26.61713116 -21.19385922]
Training loss = 5086.127
Training loss = 5086.127
Training loss = 510.231
Training loss = 320.488
Training loss = 312.175
Training loss = 311.374
Training loss = 310.892
Training loss = 310.429
Training loss = 309.975
Training loss = 309.528
Training loss = 309.088
Training loss = 308.655
Training loss = 308.229
Training loss = 307.810
Training loss = 307.397
Training loss = 306.990
Training loss = 306.590
Training loss = 306.196
Training loss = 305.808
Training loss = 305.425
Training loss = 305.048
Training loss = 304.677
Training loss = 304.311
Training loss = 303.951
Training loss = 303.595
Training loss = 303.245
```



Problem 4: The necessity of Non-linear activations

- 3 layer neural network with out activation function

input $\rightarrow$ vector x
 First hidden layer $\rightarrow W^{[1]} b^{[1]}$
 output $\rightarrow W^{[2]} b^{[2]}$
 x shape $(1, m)$
 W shape (m, k)
 b shape $(1, k)$

$$a) \quad z^{[1]} = x W^{[1]} + b^{[1]}$$

$(1, k) = (1, m) (m, k) + (1, k)$

$$b) \quad \hat{y} = z^{[1]} W^{[2]} + b^{[2]}$$

$(1, q) = (1, k) (k, q) + (1, q)$

$$c) \quad \text{from part a)} \quad \hat{y} = z^{[1]} W^{[2]} + b^{[2]} \quad \text{where } z^{[1]} = x W^{[1]} + b^{[1]}$$

$$\hat{y} = (x W^{[1]} + b^{[1]}) W^{[2]} + b^{[2]}$$

$$= x W^{[1]} W^{[2]} + b^{[1]} W^{[2]} + b^{[2]}$$

$(1, m) (m, k) (k, q) + (1, k) (k, q) + (1, q)$
 can be written as $\hat{y} = W' x + b'$

d) Show above expression

$$\rightarrow \hat{y} = (x W^{[1]} + b^{[1]}) W^{[2]} + b^{[2]}$$

$$\hat{y} = x \underbrace{W^{[1]} W^{[2]}}_{W' = W^{[1]} W^{[2]} \text{ multiplier}} + \underbrace{b^{[1]} W^{[2]} + b^{[2]}}_{b' = b^{[1]} W^{[2]} + b^{[2]} \text{ constant}}$$

$(m, q) = (m, k) (k, q)$
 $(1, q) = (1, k) (k, q) + (1, q)$

$$\hat{y} = x W' + b' \quad \text{shapes: } (1, q) = (1, m) (m, q) + (1, q)$$

e) Non linear activation functions are essential to give NN their representational power as without these activation functions, even if we have multiple NN layers like this example, they will be turned to one linear layer and will be equivalent to single linear transformation $\hat{y} = W'x + b'$. By adding non-linear activation functions, NN model can learn complex boundaries. (composition of linear maps is a linear map. A 3-layer network has same representation power of a single linear layer.)

Problem 5 - Approximating a Function with Pulses

a) Pulse 1: $(s_1, s_2, h) = (0, 1, 3)$

Pulse 2: $(s_1, s_2, h) = (1, 2, 4)$

Pulse 3: $(s_1, s_2, h) = (2, 3, 4)$

Pulse 4: $(s_1, s_2, h) = (3, 4, 3)$

Problem 5) - Approximating a function with pulses

Pulse 1: $(0, 1, 3)$

Pulse 2: $(1, 2, 4)$

Pulse 3: $(2, 3, 4)$

Pulse 4: $(3, 4, 3)$

4 pulses x 2 neurons each = 8 hidden neurons

$$P_1(x) = h [\sigma(k(x-s_1)) + \sigma(k(s_2-x)) - 1]$$

$P_1 = (0, 1, 3)$

$$P_1(x) = 3 [\underbrace{\sigma(kx)}_{u_1} + \underbrace{\sigma(k(1-x))}_{u_2} - \underbrace{1}_b]$$

Compare with
 $\sigma(wx+b)$

First $\rightarrow \sigma(kx) \quad \underline{w_1 = k} \quad \underline{b_1 = 0}$

2nd $\rightarrow \sigma(k(1-x)) = \sigma(-kx+k) \quad \underline{w_2 = -k} \quad \underline{b_2 = k}$

$$\rightarrow \underbrace{3u_1}_{w_1} + \underbrace{3u_2}_{w_2} - \underbrace{3}_b$$

$P_2 = (1, 2, 4)$

$$P_2(x) = 4 [\sigma(k(x-1)) + \sigma(k(2-x)) - 1]$$

Compare with
 $\sigma(wx+b)$

3rd $\rightarrow \sigma(kx-k) \quad \underline{w_3 = k} \quad \underline{b_3 = -k}$

4th $\rightarrow \sigma(-kx+2k) \quad \underline{w_4 = -k} \quad \underline{b_4 = 2k}$

$$\rightarrow \underbrace{4u_3}_{w_3} + \underbrace{4u_4}_{w_4} - \underbrace{4}_b$$

$(2, 3, 4)$

$$P_3(x) = 4 [g(k(x-2)) + g(k(3-x)) - 1]$$

Compare

5th →

6th →

$$g(kx - 2k) \quad \underline{w_5 = k} \quad \underline{b_5 = -2k}$$

$$g(-kx + 3k) \quad \underline{w_6 = -k} \quad \underline{b_6 = 3k}$$

$$\downarrow \quad \underbrace{4H_5}_{w_5} + \underbrace{4H_6}_{w_6} - \underbrace{4}_b$$

$(3, 4, 3)$

$$P_4(x) = 3 [g(k(x-3)) + g(k(4-x)) - 1]$$

Compare

7th →

8th →

$$g(kx - 3k) \quad \underline{w_7 = k} \quad \underline{b_7 = -3k}$$

$$g(-kx + 4k) \quad \underline{w_8 = -k} \quad \underline{b_8 = 4k}$$

$$\downarrow \quad \underbrace{3H_7}_{w_7} + \underbrace{3H_8}_{w_8} - \underbrace{3}_b$$

Sum of b: $-3 - 4 - 4 - 3 = -14$

$$\hat{f}(x) = 3H_1 + 3H_2 + 4H_3 + 4H_4 + 4H_5 + 4H_6 + 3H_7 + 3H_8 - 14$$

$$= 3g(kx) + 3g(-kx + k)$$

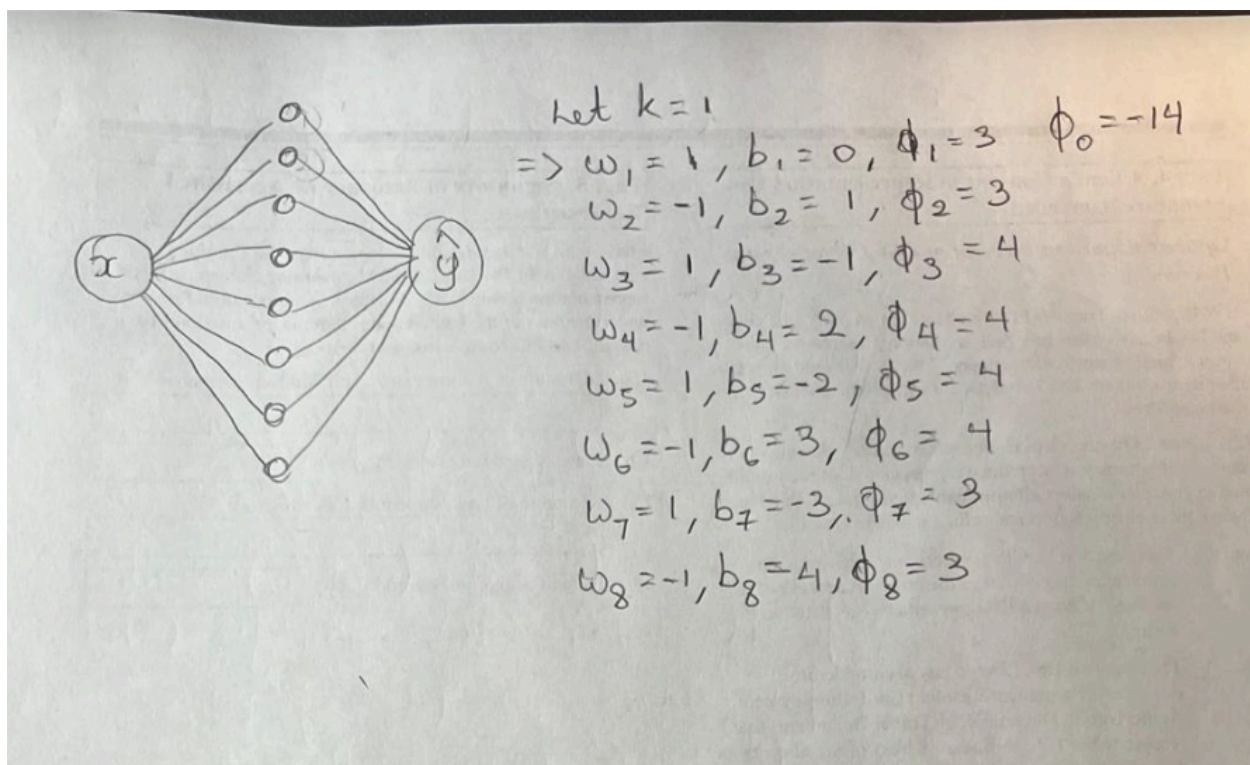
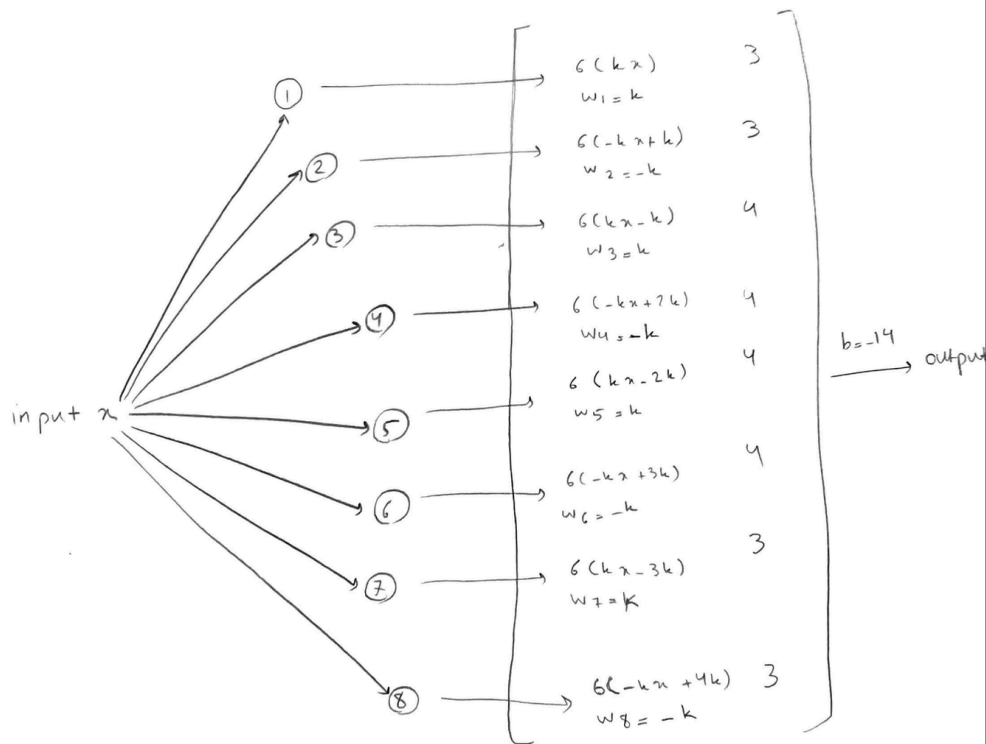
$$+ 4g(kx - k) + 4g(-kx + 2k)$$

$$+ 4g(kx - 2k) + 4g(-kx + 3k)$$

$$+ 3g(kx - 3k) + 3g(-kx + 4k) - 14$$

sigmoid

if $g(z) = \frac{1}{1+e^{-z}}$



Problem 6 - Stochastic Gradient Descent

i) $P(\text{not choosing a sample}) = \frac{n-1}{n} = 1 - \frac{1}{n}$

with replacement $\Rightarrow n$ independent events

$$\Rightarrow \left(1 - \frac{1}{n}\right) \times \dots \times \left(1 - \frac{1}{n}\right)$$

$$\Rightarrow \boxed{P_{\text{never}} = \left(1 - \frac{1}{n}\right)^n}$$

ii) $\lim_{x \rightarrow \infty} \left(1 + \frac{1}{x}\right)^x = e$; Show that $\lim_{n \rightarrow \infty} P_{\text{never}} \approx 0.3674$

$$\lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right)^n$$

$$\text{let } y = \left(1 - \frac{1}{n}\right)^n$$

$$\ln(y) = n \ln\left(1 - \frac{1}{n}\right)$$

$$\lim_{n \rightarrow \infty} \ln(y) = \lim_{n \rightarrow \infty} \frac{\ln\left(1 - \frac{1}{n}\right)}{\frac{1}{n}} = \frac{\left(\frac{1}{1 - \frac{1}{n}}\right)\left(-\frac{1}{n^2}\right)}{-\frac{1}{n^2}} = -\frac{1}{1 - \frac{1}{n}} = -1$$

$$\Rightarrow \lim_{n \rightarrow \infty} \ln(y) = -1$$

$$\Rightarrow \boxed{\lim_{n \rightarrow \infty} y = e^{-1} = 1/e \approx 0.3679}$$

iii) One random observation $\text{Var}(\nabla f(x^{(i)}, y^{(i)})) = \sigma^2$
 Let B be the batch size of a mini batch. Given B
 independent random observations, $(x_1, \dots, x_B)$

$$\text{mean } \bar{x} = \frac{1}{B} \sum_{i=1}^B x_i \quad \text{where } \text{Var}(x_i) = \sigma^2$$

$$\text{Var}(\bar{x}) = \text{Var}\left(\frac{1}{B} \sum_{i=1}^B x_i\right) = \left(\frac{1}{B}\right)^2 \text{Var}\left(\sum_{i=1}^B x_i\right) = \frac{1}{B^2} (B\sigma^2) = \boxed{\frac{\sigma^2}{B}}$$

iv) Scaling mini batch of size B by k

$$\Rightarrow \text{Var}(\bar{x}) = \text{Var}\left(\frac{1}{kB} \sum_{i=1}^{kB} x_i\right) = \frac{1}{k^2 B^2} kB\sigma^2 = \boxed{\frac{1}{kB} \sigma^2}$$

v) The claim in the paper is wrong because increasing the batch size and learning rate by a factor of k increases the variance.

Let the variance of the gradient of one observation

$$\text{Var}(\nabla f(x^{(i)}, y^{(i)})) = \sigma^2$$

The variance of a mini batch of size B ,

$$\text{Var}\left(\frac{1}{B} \sum_{i=1}^B x_i\right) = \frac{\sigma^2}{B} \text{ without changing the learning rate}$$

Scaling the mini batch size by k gives the following

$$\text{Var}\left(\frac{1}{kB} \sum_{i=1}^{kB} x_i\right) = \frac{\sigma^2}{kB} \text{ without changing the learning rate}$$

Scaling the learning rate by k , the variance of a mini batch of size kB gives

$$\text{Var}\left(\frac{k}{kB} \sum_{i=1}^{kB} x_i\right) = \frac{1}{B^2} kB \sigma^2 = \frac{k \sigma^2}{B}, \text{ thus increasing the variance}$$

To maintain the variance of $\frac{\sigma^2}{B}$, one should scale the batch size by k and scale the learning rate by $\sqrt{k}$

$$\Rightarrow \text{Var}\left(\frac{\sqrt{k}}{kB} \sum_{i=1}^{kB} x_i\right) = \frac{k}{k^2 B^2} kB \sigma^2 = \boxed{\frac{\sigma^2}{B}}$$

AI Usage

universal approximation theorem

✓ Summarized the universal approximation theorem for neural networks



The **Universal Approximation Theorem (UAT)** states that a feedforward neural network with a **single hidden layer** and a finite number of neurons can approximate **any continuous function** on a compact subset of $\mathbb{R}^n$ to **arbitrary precision**, provided the activation function is non-polynomial (e.g., sigmoid, ReLU, tanh).

Key points:

- **First proved** by George Cybenko in 1989 (for sigmoid activations); extended by Hornik, Stinchcombe & White (1989) and Funahashi (1989). Leshno, Lin, Pinkus & Schocken (1993) generalized it to *any* non-polynomial activation.
- **Mechanism:** Universality is achieved by increasing the number of hidden neurons (network *width*). Separate results show that fixed-width but sufficiently *deep* networks are also universal.
- **Formal statement:** For any $f \in C([0, 1]^n)$ and any $\varepsilon > 0$, there exist weights w_j , biases b_j , and output coefficients a_j such that
$$\sup_{x \in [0, 1]^n} \left| f(x) - \sum_{j=1}^N a_j \sigma(b_j + w_j^\top x) \right| < \varepsilon.$$
- **Limitations:** The theorem is *existential* — it guarantees a network exists but gives no practical guidance on how to find the right architecture or train it. It also applies only to continuous functions and does not imply that a single network can approximate *all* functions simultaneously.